

Inside Facebook's Engineering Culture: Part 2

A broad and deep dive in how Facebook / Meta works, from the perspective of SWEs and EMs.



GERGELY OROSZ

MAY 17, 2022 • PAID



86



13



Share



This issue is part 2 of "Inside Facebook's engineering culture." [See Part 1 of the series here.](#)

This two-part series is about "things software engineers and engineering managers should know about Facebook's engineering culture." While the information is especially useful for anyone planning to join the company, the utility of these details goes well beyond that workplace. Facebook has grown blazingly fast to achieve global dominance, meaning many of its unconventional methods might serve other tech companies hoping to grow with similar speed, too.

As a reminder, I am referring to Meta as "Facebook" throughout.

This two-part series contains more information about Facebook's engineering culture than any publication has revealed before. In writing this series, I talked with more than a dozen current and former Facebook software engineers, engineering managers, and engineering directors in verifying the details that follow. The majority of people I talked with had several years' tenure at Facebook.

Still, it's not possible to capture every single detail about an organization with tens of thousands of software engineers. Facebook gives high autonomy to both individuals and teams to decide how they work and I can't cover all that variety, but I do try to provide a useful and comprehensive overview.

In this issue, we continue [from Part 1](#), and go into:

1. Overview - covered in [Part 1](#)

- Vocabulary
- Core values
- Standout differences between Facebook and other Big Tech companies
- On the engineering-centric culture
- On the individualistic culture

2. Hiring - covered in [Part 1](#)

- Hiring processes
- Hiring support
- Software engineer hiring
- Engineering manager hiring
- Compensation: structure and examples
- Internal mobility
- Perks
- Alumni and boomerangs

3. Career

- Engineering and product roles
- Career ladders
- Competencies (axes)
- Engineering management
- Performance reviews, bonuses and equity refreshers
 - **With numbers** on cash bonus targets, equity refresher dollar values for the US, personal performance multipliers and example compensation outcomes.
- COVID and the adjusted expectations program
- The calibration process

- Promotions
- Archetypes

4. Engineering processes

- Bootcamp & onboarding
- Hackathons
- How teams and projects operate
- Documentation
- Testing
- The tech stack
- Notable internal systems and tools
- Open source
- Company divisions
- Company-wide processes
- Changes at the company over time

5. Advice

- Advice to succeed as an engineer
- Advice to succeed as an engineering manager
- Advice when leaving the company
- Advice and inspiration for founders and managers to take from Facebook

Let's dive in:

3. Career

Engineering and product roles

Engineering roles:

- **Software Engineer (SWE).** Often referred to as SWE. Within this role, you can be a:
 - Generalist: product engineer
 - Specialist: Android, iOS, Frontend UI (web)
 - Whether you are a generalist or a specialist impacts the teams you can join after Bootcamp, several of which target specialists.
- **Business Engineer:** a role where engineers work with partners external to Facebook. Roles include **Solutions Engineer (SE)**, **Partner Engineer (PE)** and **Developer Support Engineer**. While these roles involve coding, they are not as hands-on as SWEs. Total compensation for roles is around 70-85% for SWEs at the same level. At the same time, the interview process for these positions has the same hiring bar as for coding.
- **Enterprise Engineer (EE):** focusing on internal-facing tools. Their total compensation is around 70-85% of SWEs, with the base salary and bonus targets similar to SWEs, but the RSUs are lower.
- **Machine Learning Engineer (MLE).** Typically scoped to IC4+. To succeed in this loop, hires typically need an advanced degree in ML and/or multiple years of ML-specific industry experience.
- **Research Scientist.** Similar work as MLEs, but typically only hiring those with a PhD qualification.

Infrastructure engineering roles:

- **Production engineer (PE).** This role is similar to what other Big Tech firms refer to as 'Site Reliability Engineer' (SRE). [According to](#) a former PE: "The folks who bridge the gap between product and infrastructure, taking care of many and various internal tools, automating data center deployments . Not dissimilar, yet broader in scope than Google's SRE. Probably the best role in Facebook's engineering organization."
- Network Operations Engineer
- Optimization Engineer

- Hardware Engineer
- Enterprise Network Reliability Engineer. Working within the Enterprise Network Reliability (ENR) group. This group operates Facebook's data center networking.

AR/VR

- Hardware, software, research areas
- [Lots of different positions](#) are open in this area

Data science and engineering

- **Data Scientist (DS)**. Different organization than DE, but rolling up to the tech organization. I have heard of stories of people joining as data scientists, but wanting to do more machine learning engineering work. Those people had to re-interview on the MLE track to make the internal move.
- **Data Engineering (DE)**. Closer to "analytics engineering". This role involves working with SQL, pipelining and dashboarding. In the early days, this role was sometimes contentious as some people would join as a DE - which had an easier interview process. However, they would get bored and want to do more "meaty" data work that only SWEs did. To do so, they either had to move internally by re-interviewing. Failing this, people would frequently leave.

Product and Program

- **PM**: Product Manager
- **TPM**: Technical Program Manager. I cover this role in-depth in the article [What TPMs do and what software engineers can learn from them](#).

A typical product team would consist of:

- Several SWEs
- A DS
- A DE
- A PM

- For user-facing teams: a designer, and, possibly a UX researcher
- An engineering manager

One thing that surprises many software engineers joining Facebook is how much software engineers work with data scientists and data engineers, on the same team.

Career ladders

All individual contributors (ICs) are called Software Engineers (SWE). Leveling is hidden within Facebook; promotions are not public and are only between the engineer and their manager. It's frowned upon to share them.

Facebook uses the IC{number} leveling to grade individual contributor roles. For software engineers, the term E{level} is also used, where E is for engineer. These terms are used interchangeably within the company, so 'E4' and a software engineer at 'IC4' mean the same. At the same time, the IC numbering extends beyond software engineering to other tech functions like product management, design, data science, and so on.

Starting from around 2021, people have increasingly used the IC annotation over the E annotation, even internally.

We'll be using the E{level} terminology in the rest of the article. This is because our focus is on software engineers.

Here's Facebook's leveling and which titles these would map to at most other Big Tech companies:

- E3 - Software Engineer 1 (elsewhere all these levels are 'Software Engineer')
- E4 - Software Engineer 2
- E5 - Senior Software Engineer
- E6 - Staff Software Engineer
- E7: Senior Staff Software Engineer
- E8: Principal Software Engineer

- E9: Distinguished Software Engineer (still referred to 'Software Engineer' externally)
- E10: Fellow. Facebook likely has very few engineers at this level; some people I talked with guessed there are only two or three in total. This level is similar to Google's own Fellow level.

Level definitions comprise scope and three axes (direction, people, engineering excellence). Here's a summary of each level:

Level	Scope	Direction	People	Eng Excellence
E3	Task	Not expected to have Direction impact	None outside work	Follow best practices
E4	Feature	Not expected to have Direction impact	Do something small	Make the architecture better
E5	Project	Project direction. E.g. driving /scoping projects from idea to shipping	Help people around you. Mentoring is common	Improve a project, quality-wise
E6	Team + Org	Influencing others + the organization	Mentoring is expected	Improve the quality across the team and across the org
E7	Company	Organization & company direction	Contribute to large programs	Company-wide impact
E8	Industry	Company + industry direction		Industry-wide impact
E9	Industry	Company + industry direction		

Software engineer levels within Facebook.

It's worth pointing out the E5, Senior Engineer, as this is the terminal level. Compared to other companies, Facebook's expectations of Senior Engineers are usually higher.

At E5, one is often expected to wear multiple hats and do a lot of work that's not core Software Engineering work; your work could overlap with product management (PM) / technical program management (TPM) / data science (DS) and data engineering (DE)

work, depending on the role and team. There are exceptions to this: some IC5s who are very good at writing complex code on a team where this makes an impact might do much less work with PMs / TPMs / DSs / DEs.

Competencies (Axes)

Facebook has four core competencies that engineers are measured on, as part of performance reviews and promotions. Internally, these competencies are referred to as "axes":

- **Impact.** The business results, quantified.
- **Direction.** Influencing the direction of teams, organizations, the company, or the wider industry. Relevant from E5-and-above.
- **People.** Often referred to as "organizational impact". All non-technical contributions. Activities like mentoring, onboarding, being a contact person for new colleagues, team impact, community contributions, interviewing and unblocking others.
- **Engineering excellence.** Things like code quality, architecture quality, and improving engineering practices in ways that help teams move faster and/or with more stability.

Facebook has a competencies document describing all competencies and associated expectations. This document is short with a bullet-pointed list for every level. There are lots of examples linked to each bullet point, making it a live document with the linked examples being updated frequently.

On how competencies and metrics drive culture: Competencies need to be metrics driven on performance and promotions review. Basically, you need to have the numbers to show. You can't say, "this person did great work because his code is beautiful." It must be specific, such as "this person refactored 300 Java files to add null-safe annotations."

This focus on numbers can create an issue because engineers often try to work towards numerically provable solutions, rather than just making things better.

[Goodhart's Law](#) is applicable in this case: "When a measure becomes a target, it ceases to be a good measure."

An engineer I spoke with shared that they have observed a side effect whereby the focus on numbers leads some engineers to go on 'crusades', refactoring hundreds of source files semi-automatically in the monorepo. So many changes usually break a lot of stuff in the process, which causes headaches for other engineers.

I asked managers if such 'refactor crusades' do, in fact, move the needle for performance reviews. Managers shared that when these changes result in code quality metric improvements then it can, indeed, lead to a better PSC result. However, large-scale code changes rarely happen for this reason: deprecating old code is a much more common use case and not one driven by metrics.

At the same time, it's not *just* about numbers. Facebook does value things that are not fully quantifiable, but the work is still expected to help move things in the right direction. Examples include user sentiment, service reliability, improving developer experience, or the shipping of critical features, etc.

On how metrics drive people's focus: Facebook has internal tools for managers to visualize code-level metrics including lines of code (LOC), number of pull requests (PRs), diffs per PR and more. I have heard engineers complain who felt that a few managers end up focusing too much on these metrics, especially on longer projects which don't have short-term deliverables.

On how every engineer is also a product manager. Facebook software engineer [Avi Lewis](#) phrases this observation like this:

"Every engineer is also a "mini-product manager". Of course, each team has a Product Manager. That PM acts as a "mini-CEO" for the team, outlining the team vision of and prioritizing projects. However, there is a lot of flexibility for engineers to make these decisions.

There is strong emphasis on XFN collaborations - with designers, data engineers, and engineers on other teams. An engineer is required to define their own tasks, in

collaboration with the team's manager. Once done, they prioritize the list according to the most impactful projects."

On the importance of Workspace to share information on projects: Facebook uses Facebook Workplace to do most of their communication within the company. Where most companies would use email, Slack and a wiki: Facebook combines all this with Workplace.

People use Workspace to post project updates, and it's common for managers - and even senior management - to cheer on these posts, especially when meaningful milestones are hit, or business results are trending the right way.

Posting on Workspace can help bring more visibility on projects - and if upper management also sees these posts, come calibration, they might need less "selling" from managers on the impact of the projects they've seen the updates of.

There are a few internal memes on "PSC is around the corner, here come all the notes on project updates...". This is both funny, but it's also true that as PSC comes closer, people do post more frequently about project updates. These updates aim to spread the context, and make the impact of the project - and your own impact - clear to anyone who is reading it.

I talked with a few engineers who assumed that the number of likes on posts, or the number of Workspace followers could meaningfully impact their career. However, no manager or experienced engineer could confirm this. Even more, they said it's quite the opposite: the contents of posts do matter, as does bringing some awareness. But non-business-related signals like likes do not.

Workspace fills in a gap in information sharing where many other companies struggle to create visibility at things like project-level updates. At Uber, for example, we plugged this gap by sending newsletters - which had their own problems of "newsletter noise". However, most Big Tech doesn't have Workspace-like infrastructure to spread information in an easy-to-share, and easy-to-digest way.

Promotion-driven development is common among Big Tech firms and Facebook is no exception, with many people trying to game the system in order to get promoted faster.

However, in the case of Facebook, those focusing on getting a promotion usually end up helping the business move in the right direction. Read more about promotion-driven development [in this article](#).

Promotions are both about increasing the scope of your work and delivering a large impact which helps the business achieve its goals. At the same time, increasing your scope has more to do with changing your behavior to operate at the next level.

Engineering management

Management at Facebook is about supporting engineers, not managing them. Managers are trained to say "I support team X." This isn't about using politically correct verbiage for its own sake; it represents the culture Facebook expects managers to create and promote.

Levels and external titles:

- M0 - official level for junior managers. These are usually IC5s transitioning into management by leading a small team. Almost never hired externally for this level.
- M1 (same level as IC6) - 'Engineering Manager'
- M2 (same as IC7) - 'Senior Engineering Manager' Often manages large teams - up to ~50 engineers - through several M1 managers. Managers would rarely have more than 10-15 direct reports themselves.
- D1 (same as IC8) - 'Director'
- D2 (same as IC9) - 'Director'
- VP1 - 'Vice President'
- VP2 - 'Vice President'
- CTO

Individual contributor and manager levels "map" to each other like this:

Individual contributor level	Manager level
E3 / IC3	-
E4 / IC4	-
E5 / IC5	M0
E6 / IC6	M1
E7 / IC7	M2
E8 / IC8	D1
E9 / IC9	D2
E10 / IC10	VP1

The mapping of IC and manager levels at Facebook.

Compensation at the “mapped” levels is similar, so it’s reasonable to expect an E7 to earn more than both an E6 and an M1, and for an M1 to make roughly as much as an E6.

Manager competencies are designed around managers supporting engineers to grow professionally. Managers are rewarded for enabling teams. It’s in the manager's interests to give feedback that helps people grow, and to give this feedback early and often.

Promotions help managers get more senior Position IDs (PIDs). PIDs means headcount, except that PIDs are tied to levels. When an engineer leaves a team, the manager has their PID to backfill. If an E4 leaves, the manager gets an E4 PID; if an E6 leaves, they get an E6 PID.

Engineers transfer teams often within Facebook. This benefits managers who help engineers grow to the next level, who then get promoted while on their team. For example, if an engineer gets promoted from E4 to E5 and then moves to another team after the promotion, the manager now has an E5 PID they can backfill. This means they can backfill up to E5 level: so they can choose an E3 or E4 engineer as well, but not above the E5 level.

Managers also benefit from promoting engineers. This is because a track record of promoting engineers often makes manager promotions easier. This is especially true if managers set out to promote engineers to their own, current level. For example, an M2 manager promoting one or more engineers to the E6 level, or an M2 to the E7 level, helps make the case for these managers to get to their respective next levels.

Employee feedback heavily influences manager performance reviews. As part of the Pulse survey, team members rate their manager as well. For managers who receive poor Pulse ratings, this rating typically pulls their performance review downwards.

Given how Facebook emphasizes that managers are expected to support their engineers and help them grow, it should be little surprise that they don't reward managers where their team is not happy with how they operate. For managers, growing their teams and fostering a good culture is just as important as the team delivering impact.

Transitioning to become a manager from an engineer is common at Facebook, which both managers and the organization support. Transitions can happen from the E5 level and above. The M0 level was created to accommodate for transitions to the manager path.

To transition, one needs support from their manager and an opening on a team to manage.

An engineer working at Facebook recalled how their team used to "celebrate" colleagues becoming managers:

"One of the teams I used to be at would 'celebrate' someone's transition to a manager with sad reactions and 'My condolences' comments on the announcement post. The message was 'this is not a promotion, this is extra work with no change in compensation.' I don't think I've seen any other team do this, but it was pretty funny seeing it happen for the first time."

Engineering managers at the M0 level have a timeline to get promoted to M1. This is similar to how E3s and E4s are expected to get promoted. In the case of M0s, this

timeline is typically 12 months. Unlike E3s and E4s, if they are not promoted, they are not at risk of being fired; they would return to an E5 level.

Both the M1 and M2 levels are considered prestigious at Facebook, and are hard to get hired for. Many managers coming in at these levels have managed larger teams outside of Facebook, and some were even CTOs or VPEs at startups or scaleups. The M2 level at Facebook is closer to what is considered a director at many other companies.

Manager promotions are more informal than individual contributor ones. It's typical to not have Bar documents for these promotions which is typical for IC promotions - as we'll discuss in the Promotions section below.

Moving between the IC and manager path has some rules. Management moves are always lateral. This means an engineer can either get promoted from E5 to E6, or move from E5 to M0. They cannot get promoted from E5 to E6 and move to M1 in the same cycle.

Moving back from manager to be an individual contributor (IC) is not the most common path, but it happens more frequently at Facebook than it does at most companies.

When managers move back to the IC track, some of them ask to be down-leveled. For example, I heard an example of an M2 becoming an IC6 when moving back - instead of an IC7, which maps to the M2 level. This is because the manager recognized that the IC7 expectations were too high for them to succeed at, having been "out of the game" for some time.

Facebook is big on promoting managers internally, but also hires externally.

Facebook is supportive in growing managers internally, including helping engineers transition to become engineering managers. It's important to note that moving from engineer to manager is never called a promotion, but is a "transition" into management. This is reflective of how managers are not "above" engineers, and are not considered more junior than engineers above them on the parallel career ladder.

Facebook does not hire externally into the M0 level: hiring managers start at the M1 level. It's not uncommon for senior engineering managers at other companies to accept M1 offers at the company.

For M2 levels-and-above, the requirements for external hires are increasingly strict. Facebook hires plenty of people to these levels, but this is a challenging level to get offers at thanks to the numerous and high expectations.

To get promoted to M2, managing of managers is expected, as is to head up a mid-sized organization of around 3-4 teams. A track record of growing engineers to E6, and of hiring and supporting E6 engineers, can help with these promotions.

In general, to be promoted as a manager, people are expected to perform at the next level - which means delivering extra impact and scope at their current level. Manager levels have expectations defined, which are usually broad and need organizational context to interpret.

Performance reviews, bonuses and equity refreshers

Performance reviews are referred to as PSC (performance summary cycle.) Starting from 2022, they run once a year. Up until 2021, performance reviews ran every 6 months.

A more lightweight performance review cycle runs every 3 months. This used to be called the midpoint-check-in or "touchpoint". The goal of this check-in is to prevent surprises during the performance review.

With the annual performance review, "touchpoints" changed from quarterly, to every 4 months. There's a touchpoint around June, another around October, and PSC runs in January / February. In other words: there are still two touchpoints - as before - but only one PSC session, instead of the two held per year, before 2022.

The performance review and promotion review process can be done in the same meeting or is discussed separately. This is down to the organization.

Cash bonus targets in % of base salary up to 2022 were:

- IC3: 10%
- IC4: 15%
- IC5 15%

- IC6 20%
- IC7 25%
- IC8 30%
- IC9 40%
- VP 50%.

Equity refresher target is a standard monetary value per level, which is then converted to stock. They vest over 4 years.

In 2022, they were:

- IC3: \$48K,
- IC4: \$88K
- IC5: \$175k
- IC6: \$274K
- IC7: \$480K
- IC8: \$850K

In 2021:

- IC3: \$40K
- IC4: \$80K
- IC5: \$120K
- IC6: \$200K
- IC7: \$400K
- IC8: \$800K
- IC9: \$1.2M

Company performance counts for bonus targets. The targets are multiplied by the company performance multiplier. In the years between 2014 and 2021, this multiplier was typically 1.1 or 1.2. The only exception was for 2018 when the multiplier was 0.9.

Personal performance counts the most for the bonus amount, and is based on rating.

- Does Not Meet Expectations (DNE): 0%
- Meets Some Expectations (MS): 0%
- Meets Most Expectations (MM): 85%
- Meets All Expectations (MA): 100% of bonus
- Exceeds Expectations (EE): 125%
- Greatly Exceeds Expectations (GE): 165% in 2022 (was 200% in 2021)
- Redefines Expectations (RE): 250% in 2022 (was 300% in 2021)

The above performance multipliers are shared transparently within Facebook, they are published on the wiki for performance reviews.

New hires get 100% of the target until month 6. They are recorded as "too new to evaluate" during this time. Those out on longer absences - such as taking a 4-month paternity leave - are also assigned a "Meets All" for that period.

Given the difference in compensation outcomes above the MA rank, many people will hope to get beyond it. Talking with engineers and managers, it's possible to achieve EE by getting lucky with impact. However, to exceed it typically takes similar focus as preparing for a promotion does. An experienced Facebook engineer reveals:

"GE is almost always intentional (someone aiming for a promotion) and the same can be said for promotion, but EE isn't always intentional. You can end up shipping a project with more impact than expected and end up with EE instead of MA. In my view, the delta from EE to GE is much, much larger than from MA to EE."

Compensation changes are based on personal and company multipliers:

- **Cash bonus** = Target * personal multiplier * company multiplier
- **Base salary increase** = (company-wide target) * personal multiplier * company multiplier
- **Equity refresh** = Target * personal multiplier * company multiplier

GE and RE all mean large compensation outcome differences and it should be no surprise that it's difficult to achieve these ratings without delivering outstanding impact. Tenure, having a strong network and working well with the business usually all greatly help in standing out this way.

Example compensation outcomes are easy enough to calculate, thanks to the formula of these being clear. Let's run a few in a year when the company multiplier is 1.1:

- E3 meets some expectations: no cash bonus, no equity refresher.
- E3 redefining expectations: 33% cash bonus, \$132K equity refresher (vesting over 4 years)
- E4 meeting most expectations: 14% cash bonus, \$75K equity refresher
- E5 greatly exceeding expectations: 33% cash bonus, \$264K equity refresher
- E6 meeting all expectations: 22% cash bonus, \$220K equity refresher
- E6 exceeding expectations: 27.5% cash bonus, \$275K

The risk of meeting "only" most expectations is real. Facebook is a culture of high performance and while at many places meeting "most" expectations is acceptable, at Facebook getting MM is a warning. Every year, roughly 10-15% of the workforce at the E3-E5 levels get MM, according to managers at Facebook I've talked with.

An E5 shares how they feel about this pressure:

"A lot is put on your back - you're expected to solve problems yourself in a way that some people might not see as fair. One typical thing to hear from managers is "As an E5, you're expected to do X even if X means "read between the lines of this specific cultural, political thing at Meta that you have no experience or context with". The implication being that if you don't, you might get a bad review, which can lead to eventual termination."

Getting two MM ratings in a row results in the employee first being put on a coaching plan, and - if there is no improvement - being put into a PIP, a Performance Improvement Plan.

PIPs are done differently than [how they are executed at Amazon](#). At Amazon, managers have little incentive to help employees clear a PIP, thanks to internal targets for unregretted attrition.

At Facebook, the company is invested in helping the employee improve, as are the managers. The manager, HR and the employee jointly make a plan for their PIP. A large portion of people on a PIP clear it and go on to have a successful career. I talked with an engineer who was put on a PIP as an E3 entry-level engineer, cleared it, and then got promoted several times, up to E6 at Facebook. Here is what they share of their recollection of PIP:

"I joined Facebook from a startup and had no idea how a company like this worked. I honestly thought I was doing great, cranking out code.

In my performance review I got a cold shower. My rating said I didn't ship anything with impact, my engineering excellence was non-existent. I have to add, my onboarding was also subpar back then, as I had no feedback until this performance review, and no mentor.

I was put on an official PIP. My manager made a plan that was fair. I went through it and completely changed my mentality from there on, being aware of the competencies, and started to ask for feedback more regularly. I adjusted quickly to the Facebook culture and went on to have a good career.

My PIP was many years ago. These days, onboarding is much better: people have onboarding buddies and mentors. Expectations are also communicated more broadly. I don't see people get on PIPs as often as back in my day."

COVID and the adjusted expectations program

When COVID started in the spring of 2020, Facebook gave everyone an EE (Exceeds Expectations) rating in the middle of 2020. Following this one-off skip on performance reviews, they put in place a program in place to allow for people who are performing below their expected level to keep their job. The company calls this program "adjusted expectations". It works like this:

Before the start of the half, the engineer informs their manager about wanting to utilize Adjusted Expectations. The two of them then sit down and draft a document that clarifies what are the new, reduced, expectations. This document gets approved by the manager, and, possibly, the skip level.

Then, as long as the person is able to hit those reduced expectations, they get a MA (Meets All) rating in the PSC.

The catch is that those embarking on Adjusted Expectations are not eligible to get above MA ratings: so no EE, GE, or RE.

The company discontinued Adjusted Expectations at the end of 2021 - meaning that from 2022, performance reviews are back to normal.

The calibration process

The calibration process for performance reviews is fairly standardized. Managers gather to calibrate engineers, accompanied by HR. They separate people based on performance in the different performance buckets: DNE, MS, MM, MA, EE, GE, RE.

Calibration typically happens at the org-level, led by a director. An organization usually has 100-200 engineers. For smaller organizations, they are typically calibrated together with their parent organizations.

E7+ levels typically have their own calibration. Each level has different groups for calibration:

- E7 calibration is typically done by combining several organizations
- E8 calibration is typically done at the VP-level
- E9 calibration is usually done at the company-level

At E7+ levels there are usually 20-30 engineers in each calibration which the group goes through.

To prepare for the calibration, managers put together a "calibration pitch" for each of their engineers. This pitch consists of:

- A detailed summary of the engineer's past 6 months (starting from 2022: it's the past 12 months)
- Rating proposal and reasoning for this rating
- Promotion suggestion, if the engineer is up for promotion

The calibration process typically works like this:

- The manager for the engineer starts with a concise summary, sharing this in 1-2 minutes.
- Follow-up questions from other managers in the room.
- The goal of this session is not to grill the manager or criticize the engineer. The goal is for everyone in the room to get on the same page about what a reasonable rating proposal is.
- The explicit goal of this session is not for managers to "fight" for the best possible ratings for their directs. It's to ensure that all engineers are fairly rated given their contributions, and for managers to understand why people are rated higher or lower.

Some organizations experiment with different calibration formats. One of these is doing calibration only in written form, in order to reduce bias that can happen during in-person back-and-forths.

Calibrations are often observed by guests. Guests can include managers from other organizations helping with cross-calibration across the org. E7+ engineers – especially those playing tech lead roles – often tend to attend calibrations as observers.

Facebook does not have overly strict stack ranking targets. Unlike the infamous Microsoft forced stack ranking that the company used [until 2013](#) – where employees were strictly stacked as 10% below, 70% meets, 20% exceeds – Facebook doesn't enforce such rigid rankings.

There are always high-level guidance numbers for calibration, which have to do with budgets. For example, in a given year there might be a guidance of 5-10% for DNM and

MS, 10-15% for MM, 25-35% for MA, 3-5% for RE. The actual numbers will differ based on the organization, and the budget.

The budgets are defined within ranges, and they do not necessarily need to keep strictly to the organization's level. If needed, organizations can go above or below the recommendations. If the organization doesn't hit the target for good reasons, they can escalate to the VP for an exception. I talked with a long-tenured manager who shared how one year their organization had gone out of bounds, applied for a VP-level approval and got it.

The VP-level typically means 1,000-3,000 engineers. At this level, these ranges typically need to be met. If the range is not met, the VP would typically need to ask for a C-level exception, as going out of bounds has budget implications.

Promotions

Promotions happen every 6 months. Even as the PSC cycle has been extended to every 12 months, Facebook has kept in place the cadence of promotions every 6 months.

Getting promoted to E4:

- E4 is still considered a somewhat junior level.
- The process is lightweight and simple, in-line with how this position is not yet for a senior engineering level.
- E3 engineers who get things done, ship projects in-line with expectations and keep growing tend to be promoted to the E4 level in about two years.

Getting promoted to E5:

- The promotion criteria are a bit more vague for E5 than it is for E4.
- **E5 is a terminal level.** This means that there is a pressure to get to this level, but not to go beyond it. More on this to come.
- The promotion process is more strict than for E4, given the terminal level. However, it's still much more lightweight than promotions in general are at other Big Tech companies.

Getting promoted to E6 & above

- The rules for these promotions are specific to the organization within Facebook. This means that, for example, in the WhatsApp organization there might be a somewhat different process than within the Messenger organization. The biggest difference comes down to how much formality each organization applies to this process.
- In some organizations, two champions are needed who are both at IC6s or above. The IC6s cannot be from your org, in order to demonstrate cross-organization impact. In these organizations, to get to the E6 level, you need to work across other organizations with IC6-and-above people; engineers, product managers, data scientists, managers, and so on.
- A promotion document is needed in most orgs for E6+ promotions. In the more formal orgs this document includes:
 - Bar document: written with the manager, outlining overall expectations.
 - "What is the bar": a section of this document, defining what goals this person should be achieving to prove they are at the next level.
 - This document outlines goals and is typically written about 6 months before the promotion is due. It's updated before the promotion, with the results.
 - A committee reviews the promotions documents and makes a promotion recommendation.
- Several managers I've talked to mentioned that in general, impatience to get promoted is a signal a person is not ready for promotion to E6-and-above levels.
- **For E7**, a common expectation is to move a critical metric for the organization by a significant amount. This means pitching the project, gathering resources, executing, delivering and measuring results.

"Get promoted to E5 or get fired" is still a concept at Facebook. It's expected all engineers get promoted at least every 2 years to get to IC4 and another 2.5 years until they get to IC5. Those who do not get promoted intime face pressure to perform at the IC5 level from that point on. There is no such pressure after the terminal IC5 level.

For nuance, the deadline is not a hard one. Once people hit the deadline, they are calibrated at the next, IC5 level. The closer people get to the deadline, the higher the expectations are at PSC. This means that if people don't start performing at the next level once they are past the deadline then their ratings will be "Meets Most" at best. Getting such ratings means being put on a PIP. Failing that PIP means they are let go. However, it is possible to get promoted past the deadline by proving next-level performance.

Google has similar expectations for getting promoted to L4. Uber used to have this expectation to get promoted to L5 until 2016, when the company removed this pressured expectation.

Promotions are not public at all. A promotion is shared only between the person promoted and their manager. It's considered bad etiquette to mention being promoted to others. The title for engineers is SWE whether they are an E3 or E10. Same goes for managers where their title is Engineering Manager, regardless if they are M0, M1, or M2.

A downside of concealed leveling is how it makes it hard to evaluate other people as part of the PSC cycle. You could find yourself in a situation of giving feedback saying "this person is awesome" thinking they're doing great as IC4. However, in reality they might be at IC6 and barely meeting expectations.

Archetypes

Starting at the E6 level – mapping to Staff Engineer at most of Big Tech – expectations become more ambiguous in all engineering career ladders. To make dealing with this ambiguity easier, Facebook introduced a concept that is unique among Big Tech firms called "archetypes."

Archetypes describe an engineering profile at E6 or E7 levels and above. These archetypes are then used both for promotions and for performance reviews. They are useful as they make it clear what the focus areas for each archetype is, and which areas they might have less output on.

Facebook has the following archetypes:

- **Generalist.** The "default" archetype. They lead projects end-to-end.
- **Specialist:** expert in a very niche area. They might be specialists within their organization (E6 & above), the company (E7 & above) or the industry (E8 & above). They usually help with lots of projects, but tend to do few standalone ones.
- **Coding Machine:** someone coding a lot and who solves problems few engineers can. They are not a tech lead. However, they attract other coders and multiply their output.
- **Tech Lead.** Leading efforts across the organization, company or industry.
- **Fixer.** Someone with deep domain knowledge who debugs and fixes complicated problems, improving critical systems as they do so.
- **Product Hybrid.** They solve vague and complex business problems that require a product / engineering solution.

Archetypes don't have standalone requirements lists. However, this approach helped create a shared vocabulary for both managers and engineers when talking about people in performance or promotion context. For example, saying "this person is a specialist archetype" implies it's no wonder they are not leading large projects, like a Tech Lead archetype does.

The first archetypes were reportedly introduced by [Kent Beck](#), when he worked at Facebook between 2011 and 2018.

The Coding Machine archetype was created for one specific engineer at the time.

I talked with former Senior Staff Engineer [Michael Novati](#) – now cofounder of [Formation](#) – who shared how this archetype came about because of his promotion. He explained:

"Facebook takes fairness as the #1 consideration. I was basically the "largest volume" committer at the company, but when they calibrated me against the E7 in the other areas – which granted was a very small number of people, about 100 – they didn't feel like I was as impactful, or on par.

I was pushing hard to have my contributions recognized more. I did also do 300+ interviews at that time, and worked on important things on the side. My

understanding is that my Director of Engineering at the time wrote up a Coding Machine archetype and presented it to the calibration group that reviews all E7s at the company. The committee accepted this, and added the archetype to the list of archetypes.

Once at the E7 level, I performed fine. However, I was still pushed to have a broader impact than JUST raw output. For example, I organized a Code Health summit for people to present on impressive cleanup efforts. Still, I wanted to just be in the code more!

I had a path to continue to E8. This would have required me to motivate and coach more engineers to become Coding Machines themselves and grow as an engineer in this direction."

4. Engineering processes

Bootcamp & onboarding

Bootcamp is Facebook's onboarding program. This program goes well beyond what any other Big Tech company offers. Bootcamp is a 6-7 week-long program mandatory for all engineering hires. The goal of this onboarding is to learn about Facebook's technology and engineering culture, for engineers to try their hand in fixing small bugs or building small features, and to choose their team.

Managers, directors, and senior ICs all go through the Bootcamp, as well. However, they often get to choose their team before they join the company, so they do Bootcamp without the pressure of team selection.

During bootcamp, engineers choose a focus area:

- Backend / systems
- Web
- Android
- iOS

- **ML.** This is mostly accessible for those interviewing for the ML, AI, CDS, and AR / VR, Computer Vision pipelines. While people can still attempt to join an ML team coming out of Bootcamp – even if they did not complete the ML Bootcamp – there are few teams that end up hiring folks this way.

Several engineers I talked with recalled how Bootcamp felt like an overwhelming firehose of information. This was partially by design.

Team selection. During Bootcamp, engineers both reach out to teams, and teams with open headcount reach out to them. At the end of Bootcamp, engineers choose a team, from teams that indicate they have an opening and are willing to take this engineer on.

Depending on the location of the engineering office, team selection can be overwhelming. In smaller offices with 10-15 teams, it's easy to choose a few that a new engineer is interested in. However, in larger offices such as the Menlo Park headquarters (MPK), it's not uncommon for Bootcampers to get 40-50 reach outs. This can be a double-edged sword as it presents new recruits with several options, but can also make it pretty difficult to evaluate the options and make a decision.

It is common to have a "speed dating" setup with team matching. An engineer shared how things worked around 2018 in the London office:

"In 2018, I did 5 weeks of bootcamp, followed by 3 weeks of 'speed dating' team selection. This last part meant sitting with a team, picking up some easy tickets, seeing if you're interested, and then moving on to the next team. I had to choose my team by the end of week 8.

This approach worked well as I could find out what the actual work on the team was like. Teams invested a lot to make sure they were friendly to bootcampers. For example, most teams had a variety of easy tickets that bootcampers could pick up.

As a new joiner, I really enjoyed this 'speed dating'. It felt like several teams bent backwards to get me to join their team."

Team onboarding. After Bootcamp, the new hire joins a team. Every team or organization has their own – typically far less formal – onboarding process.

Not just for engineers. Bootcamp is not just for SWEs. All people in the tech org - like PMs, designers, data scientists - go through Bootcamp. However, outside engineering Bootcamp is not as hands-on.

As an example, the product manager bootcamp is typically 4 weeks long, with each week having a theme of skills to learn. These include working with engineers, marketing, and design. Product managers going through this bootcamp spend a lot of dedicated time learning how Facebook builds products.

Hackathons

Facebook is known for its internal hackathons. Facebook engineering director Pedram Keyani has been central in starting the "Hackathon movement", organizing [the first few dozen hackathons](#). In [a 2012 Facebook post](#), here's how he described hackathon:

"Before every hackathon, we create an internal wiki where people can start brainstorming, plug their hackathon ideas, and find teams to help them execute. Hackathon projects are usually driven by cross-functional teams, with everyone from engineers to lawyers to UX researchers, so the wiki and event wall are usually filled with comments like, 'Anyone want to code with me?' or 'I can code but my design judo is weak, want to design project X?'

A lot of people also use hackathons as a time to hone a new skill or work on projects where they can be exposed to unfamiliar technologies. Everyone, regardless of team or skill level, is free to check out which projects are happening and to jump in. (...)

There are only two rules: you have to work on something outside your day job, and if it's your first hackathon, you have to hack.

After each hackathon, we keep the momentum going by holding a prototype forum where everyone who built a project can present it to the company. The prototype forum usually happens a week after the hackathon, so it gives people a chance to fine-tune their projects and prepare them for live demos. Everyone gets the same amount of time at the forum – two minutes – to convince their peers that their idea should ship."

Hackathons are important at Facebook for a few reasons:

- They're seen as a way to get people together to solve a problem.
- It's an opportunity for others to try working with a different group, and see if they like this dynamic. After the hackathon, engineers who enjoy working together might find ways to work on the same team together.

Several Facebook products ideas were prototyped during a Hackathon. Facebook Groups, Safety Check and Facebook Chat were all originally a hackathon project and got funding to turn into a full-blown product following their respective hackathons. Hackathons also inspired features like [Facebook Live](#). Of course, many hackathon projects never make it into a Facebook product, but it's questionable if the former products would have been built without the prototyping happening at Hackathons.

Hackathons continued to be important during COVID, and after the crisis subsided. For example, this year the London HQ already hosted a hackathon, and more are on the way across offices.

Facebook regularly runs intern hackathons as well. These events typically run a few weeks ahead of the "regular" ones.

Related articles:

- [Facebook's hackathon culture](#) (2011, Forbes)
- [Inside Facebook's 32nd hackathon](#) (2012, Wired)
- [Facebook executives on the importance of hackathons](#) (2017, Business Insider)

How teams and projects operate

Teams have full autonomy on how to operate, and they make full use of this. Every team operates differently as a result. This level of autonomy and the lack of any standardized processes stand out, even in Big Tech.

To organize their work, teams tend to either use Phabricator, or the Task tool – an internal ticketing tool – and a decreasing number of teams use Asana.

Don't expect to see Scrum or other, heavyweight processes. Nothing stops teams from using Scrum, but just like [most of Big Tech avoids Scrum](#), the majority of Facebook engineering teams do, too.

Engineers lead many projects. This might seem strange for engineers coming from places where engineers are not as product-oriented, but the setup works well. Here is how Facebook PM Ben Erez describes how projects are run with a short summary:



Read more on [how software engineers can run projects](#).

Rolling out and monitoring systems are unique to products and organizations. There are dozens of internal tools to roll out across the Core Facebook product, to Messenger, to web, to mobile, and so on.

Experimentation tooling and analysis is beyond excellent, compared to other Big Tech firms. The ease of rolling out experiments, and the amount of actionable insights engineers get from doing so, is unmatched across companies operating at a similar level.

Oncall and handling outages are left to the team to manage, just like any other process. One of the few standardized approaches is related to recording of incidents, and assigning severity to them.

Severity levels for incidents - called SEVs, supposedly from coming from the name "Site Event" - provide a way for teams to organize troubleshooting and communicate internally. With SEVs, teams analyze the root cause of the problem, make preparations to prevent similar incidents in the future and share the knowledge.

Severity levels are as follow:

- **SEV0:** One or more critical sites are down. The [2021 Facebook outage](#), when all of Facebook, Whatsapp, Messenger, Instagram and Oculus went down for 7 hours, was a SEV0.
- **SEV1:** "There's a large fire!" Urgent and very critical.
- **SEV2:** urgent, and critical.
- **SEV3:** urgent, but not critical.
- **SEV4:** not urgent, not critical.

The meeting load can be high for engineers. Talking with senior engineers, they shared how they observed a meeting load within Facebook that was higher than at similar companies. As a former engineer shared:

"The meeting load at Facebook hwas high. At IC5+ so many engineers would complain that they barely had time to write code as all they did was sit in XFN meetings. Being pulled into more meetings is common at other companies as well, as you get senior. However, at Facebook, it got to the point before 2020 where it was very hard to find a meeting room to book a meeting, as they were all constantly booked out."

Documentation

Documentation of systems or code varies heavily per team. In general, the effort to document is low, save for a few key systems. Pointing to the code as the source of truth is a common approach. Teams have the autonomy to decide how much they document.

If documentation helps teams move faster, or to reach their goals better – such as adoption with open source projects – then they will document more.

The code is used as the source of truth when there is no additional documentation. The benefit of this approach is there is only one source of truth and there is no outdated documentation. The downside is that finding answers to questions not obvious from the code means looking through changelogs, or tracking down people who touched the code and can explain decisions.

Solving problems via code is the preferred approach across most of Facebook. This is the mentality for a few reasons:

1. Teams are receptive to diffs from other teams. Suggesting a code change fast-tracks the conversation. Instead of discussing proposed code changes back-and-forth, just propose the change and have the discussion about this change on the diff itself.
2. It's how to make things scale. Given the size of Facebook, relying on people to solve problems means there will be bottlenecks, mistakes, and so on.
3. The engineering-centric culture. As engineers are empowered and have a larger say than most companies would have, it's natural to turn to code as the way to solve things.
4. The experimentation tooling enables the results of changes to be validated easily.

Testing

A major standout difference with Meta versus most of Big Tech, is how few automated tests it has. This approach is strongly correlated with the "Move Fast" value and how Facebook's development approach has always been metrics-driven.

When Kent Beck, the creator of Extreme Programming and a major advocate of Test Driven Development, joined Facebook in 2011, he was aged 50. He thought he had seen everything in the industry. Here's his first impression of the company after he finished Bootcamp, from his interview [on Software Engineering Daily](#):

"I see this place, it's crazy. It looks like a clown show and yet, things are working really well. They weren't doing the things in my books. (...)

In the back of my mind there is the mystery of this bumblebee. In theory, this process should be a disaster. In practice, it's working extremely well at doing two things at the same time: at scaling, and at exploration."

After he joined, during the first Hackathon people could host classes for others. Kent hosted a class on Test Driven Development. He saw people were not using TDD, so he expected people would show up to learn from him. The Argentinian Tango class was full, with a waitlist. Zero people showed up to Kent's class on TDD.

In the early days of Facebook, testing was uncommon for valid reasons, though. In the same interview, Kent Beck shares his observations on why Facebook worked fine even with no automated tests, back in 2011:

1. **Many problems only show up in production, not via tests.** In those days, most of Facebook's production issues were related to the production environment, and tests did not always help. So Facebook decided to not bother with them.
2. **Code review.** Facebook always had code review in place, from the earliest days. Though no replacement for tests, this also meant that every change had a second pair of eyes look at it.
3. **Automatic canarying and alerting on the canaries.** Facebook was one of the first companies which implemented canarying at scale; shipping code changes to a small number of machines and gathering feedback on key metrics there. Even though they had no unit tests, this step gave quick feedback when something looked wrong.
4. **Logging and operational awareness core to the culture.** Engineers would get feedback post production on their features. This type of feedback is uncommon even today at most of Big Tech.

Unit tests getting in the way of "Move fast" was the major reason for them not being adopted. Kent Beck tried to do thing properly with tests and here's what happened:

"In Bootcamp, you're supposed to put code into production the first week. I was very careful to write tests and do everything properly.

I got in a fair amount of heat, because my first feature didn't land for three weeks. People are like, 'Man, I don't know how this is going to work out.' Well, I was wondering that, too. I had a huge case of imposter syndrome when I landed at Facebook and realized just how different everything was.

Then the tests I had written broke almost immediately. They were deleted. That was one of the things that surprised me.

If you had a test and it failed, but the site was up, they just delete the test. If you had tests that were intermittent, that were non-deterministic, they were just deleted.

At first, I was shocked. Like, delete a test?

But this test was producing noise and it was not producing signal. If you eliminate this noise production, by definition the situation is clearer all of a sudden. The fact that you wish that you had a test for something, well you don't have one. Just chuck it and let's move on."

Automated staged rollouts into layers of staging environments are world-class within Facebook. While Facebook has not been big on testing, they have doubled down on catching issues before they hit production.

The "core" Facebook product has an impressive setup of staging environments for the main repositories. Code deploys go through deploys to about four layers of deployments before going into production: all of this is automated.

The layers start with a testing environment with automation, progress through an environment all Facebook employees use, go through a test market, and the final environment is the global Facebook service. In each layer, if an issue was found - either metrics off, or someone flagging a problem - then the release would be halted in progress.

Here's how a former Facebook employee shared a story about an Ads issue being caught well ahead of causing an outage:

"I was working in Ads and I remember this particular bad deploy which would have made a massive financial loss to the company. The code change broke a specific ad type.

This issue was caught in stage 2 out of 4 stages. Each stage had so many health metrics monitored that if something went wrong, these metrics would almost always trigger - like this first safety net had done so."

All Facebook employees used the before-production builds of the Facebook website and apps. This means that Facebook has a massive testing group of ~75,000 people testing their products before they go into production, and flagging any degradations.

Testing is more common at Facebook today than in the past, but it's still far behind most of Big Tech. Talking to engineers, tests are typically optional on most non-critical products. Not writing a test for a feature change is usually not a reason to reject a diff for many teams.

What is not surprising is that infra teams are the ones which typically have the most strict approach to tests. Product teams are typically more easy-going. Of course, much depends on the product's maturity, and whether issues are caught by other means like automatic canarying or monitoring and alerting.

Teams operate with autonomy and there are a growing number of teams on which testing is not optional, and some enforce full unit test coverage. Engineers are also empowered to challenge their team's status quo. As a current Facebook engineer shared:

"I joined a team with very ad hoc engineering practices. Then a few engineers took the initiative and landed major improvements the next two years, and they also saw promotions follow. So there's always some room to do better if you really want to.

The focus on testing does vary based on the team, though. If you really care about testing: join a hardcore infra team."

On one hand, I'm surprised that Facebook still treats tests as less important than most of Big Tech. On the other hand, Facebook is unique in how much feedback they can get from their users – most products have above 1B users! – and how they have world-class infra for automatic canarying, alerting and a support for operational awareness.

Facebook does not have a QA function across the company. Individual teams might employ QAs, especially for larger products. The teams I talked with typically worked with QA in testing unreleased features as manual and exploratory testing.

This approach is similar to how most consumer-facing Big Tech does not employ QA engineers in large numbers. SWEs are responsible for building automation – which usually means unit tests at most, and perhaps some integration tests. At Facebook, platform teams typically have more automating testing in place, while product teams can opt to have far less.

What counter-balances for the lack of QA – and more bugs slipping into production for users to discover – is the automated canarying, close-to-real time metrics and the ability to change production code quickly. As a Facebook software engineer reflects on the culture shock, coming from a company with a dedicated QA function:



Andrey Goncharov
@ai_goncharov

[@GergelyOrosz](#) It blew my mind at first at Meta. Only later I realized that it works here only because we also push to production fast. Even if you break anything, you can pull the false version instantly and push a fix in a matter of hours.

8:04 AM • May 16, 2022

I don't suggest that most companies should follow the same testing approach as Facebook. However, I will suggest that if you're in a similar situation – where other things can substitute for tests, or when small and frequent bugs don't harm your business but moving faster than your competition is essential in gaining market share –

then you might consider experimenting with less common testing approaches, like Facebook did in its early days.

The tech stack

The tech stack is non-traditional, custom and quirky:

- [Hack](#) is used on the backend and on the web.
- **iOS and Android** approaches and tooling are non-traditional.
- **Toolchains** are custom, as are many of the compile tools.

You are in for a lot of reading and learning when you join Facebook, hence Bootcamp being mandatory for all new recruits.

Visual Studio Code is the integrated development environment (IDE) most engineers use, currently. Facebook uses a customized distribution of this tool. It has built several internal plugins to make work more seamless. There is a push to make it a no-brainer tool of choice for all Facebook software engineers to use this tool by the end of 2022. As with all tooling, Facebook never requires engineers to use a specific tool, and this is no different to VS Code.

Before 2020, most Facebook engineers used Atom, especially those working with the www monorepo. Back then, Atom was the officially supported tool, especially thanks to the internally developed – and later open-sourced – [Nuclide integration](#). Nuclide provided a first-class development environment for React Native, Hack and Flow projects.

Most developers work with a remote server, not locally. Starting from around 2019, all web and backend development [is done remotely](#), with no code copied locally, and Nuclide facilitating this workflow. In the background, Nuclide was using virtual machines (VMs) at first, later moving to OnDemand instances – similar to how [GitHub Codespaces](#) works today – years before GitHub launched Codespaces.

Mobile development is still mostly done on local machines, as doing this in a remote setup as with web and backend has tooling challenges.

Facebook's monorepo is worth being aware of. Facebook was one of the first companies to move all its source code into one, giant code repository. As Facebook wrote in 2014 in the article [Scaling Mercurial at Facebook](#):

"With thousands of commits a week across hundreds of thousands of files, Facebook's main source repository is enormous – many times larger than even the Linux kernel, which checked in at 17 million lines of code and 44,000 files in 2013. Given our size and complexity — and Facebook's practice of shipping code twice a day, improving our source control is one way we help our engineers move fast."

Since 2014, Facebook has now several monorepos: the additional ones include mobile monorepos for iOS and for Android.

Technically, Facebook has one megarepo, and within this megarepo it has several monorepos::

- **www**: the core of Facebook. This monorepo has separate www, ios and android folders. These folders have separate checkout profiles, so engineers usually check out only the monorepo for their specialization.
- **Other products**: WhatsApp, Instagram, Messenger have their own folders.

As an early pioneer of the monorepo workflow, Facebook built and open-sourced [Buck](#), a build tool used by many other companies with monorepo setups.

Development is done on only one branch. Facebook follows trunk-based development where engineers merge changes into the main branch, and this main branch is pushed to production immediately after.

On one end, this approach keeps the codebase clean. On the other, thanks to how popular codebases can have hundreds of changes per day, it means engineers often have to rebase and resolve conflicts to land their code.

Notable internal systems and tools

Facebook builds pretty much every developer tool in-house. This approach comes from the early days, when there were few suitable vendors Facebook leadership felt

could support the scale and growth of the company.

Also, it should not be surprising that a company which creates a new language - Hack, described as "an object-oriented programming language for building reliable websites at epic scale" - would rather build an in-house implementation of an incident alerting system like PagerDuty as opposed to paying for the vendor, integrating it, and finding out that the vendor does not support edge cases they need, or cannot guarantee the reliability Facebook is looking for.

Facebook Workplace is central to how Facebook works. It's hard to capture just how much of a central system Workplace is across Facebook. Think of it as Slack + wiki + documentation + social all in one.

Workplace has plugins for many other Facebook tools. For example, people can type "#fileabug {add details}" and this would automatically create a bug ticket.

Engineers are encouraged to share about their work on Workplace, to gather feedback this way, and to share project results and impact through Workplace. This posting creates more transparency, sparks discussions that can later be referenced, and forces [software engineers to become better writers](#).

Getting added to the right groups relevant for your work is important: similar to Slack groups, you will not see content specific to groups otherwise.

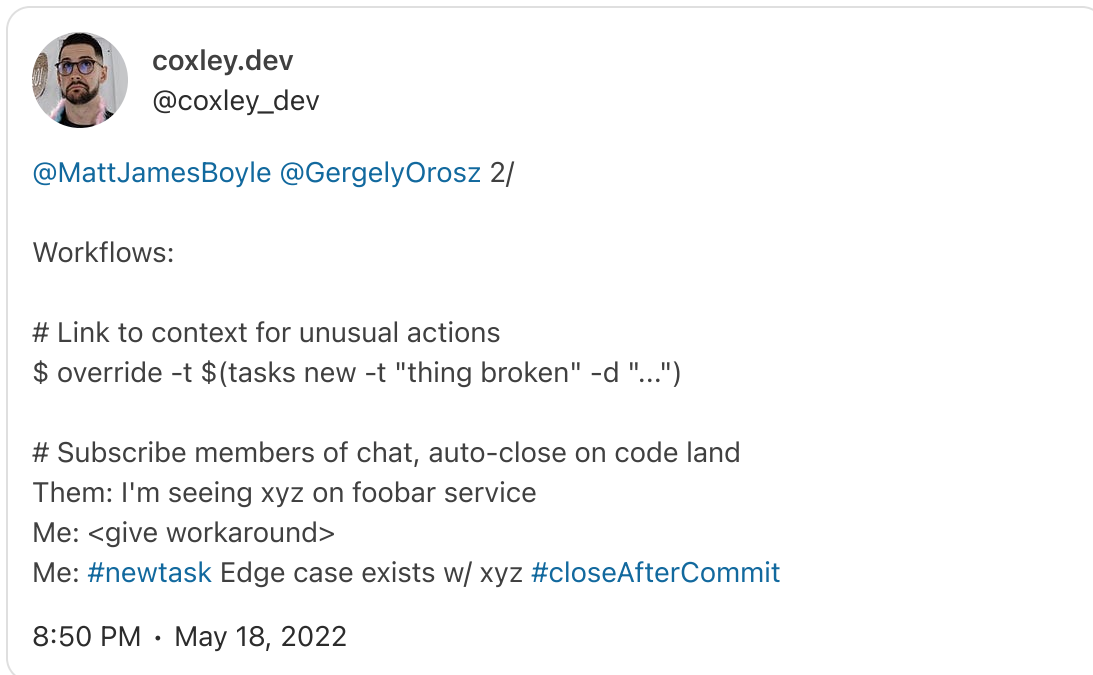
Tasks is a frequently used, lightweight internal project management tool. To create a task, you only need to add a title. You can reference tasks as dependencies. Mentioning a task on a diff / pull request automatically links it.

Tasks only has the concept of tagging. There are no projects, spaces or other concepts. Most teams use a different tag for each project. Tags can be added with square brackets in the title, like "[project_x] Task name".

Tasks supports scheduling, but most teams don't use this feature.

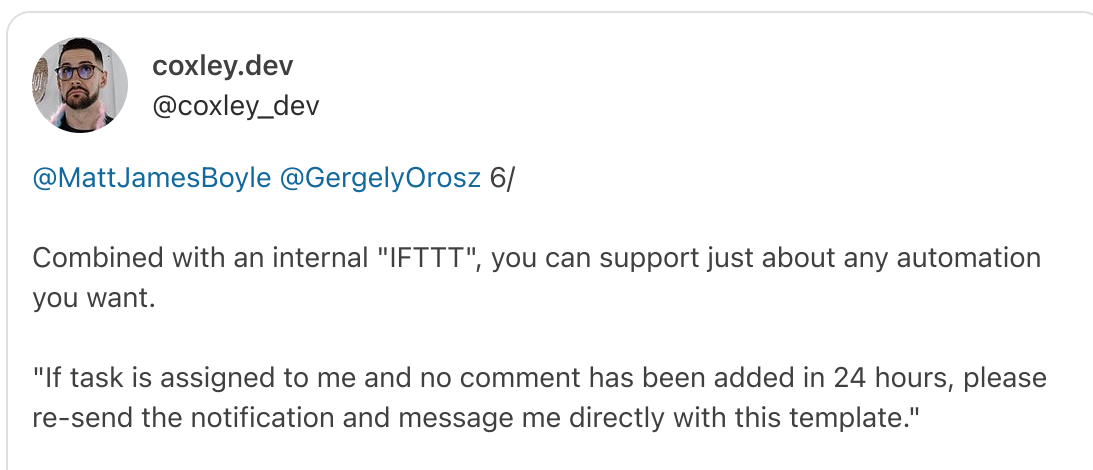
Tasks support workflows common in engineering teams like adding more context for unusual actions, or subscribing members of a chat to a task, and automatically closing

the task upon landing the diff that mentions it. As described by a former Facebook engineer:



Tasks differs from most existing ticketing tools in how it was not built with top-down the goal to support all use cases. It's the opposite: engineers built it bottoms up, adding the most common use cases Facebook engineers want to solve for.

Butterfly is a very useful generic rules engine. It's a bit like "Zapier or IFTTT but for Meta internal systems". Automate anything from Diffs, Tasks and other internal tools. For example, you can set up to get a notification for a code change request made by someone outside the team. Here's how a former Facebook engineer describes it:



9:02 PM · May 18, 2022

Other internal tools Meta uses which are worth mentioning:

- **Asana:** some teams use it for tickets.
- **Phabricator:** Facebook's version of GitHub + JIRA for engineers.
- **Diffs:** code changes, CI, unit test, linting.
- **CodeHub:** code search. All repos searched. It has fancy features such as searching for recent code changes.
- **Gatekeeper:** an experimentation tool; one of several.
- **Experiment:** an experimenting tool with plenty of hand holding capability. Helps run experiments and to see tier results.
- **ODS:** Operational Data Store. Used most commonly in alerts and dashboards for trouble-shooting systems with 1-5 minutes of lag. Stores ~2B time series counters and has a load of about 40,000 queries per second.
- **Scuba:** Facebook's fast slice-and-dice data store. It stores thousands of tables in about 100 terabytes in memory. It ingests millions of new rows per second and deletes just as many. Throughput peaks around 100 queries per second, scanning 100 billion rows per second, with most response times under 1 second.
- **Hive:** Facebook's data warehouse, with 300 petabytes of data in 800,000 tables. Facebook generates 4 new petabytes of data and runs 600,000 queries and 1 million map-reduce jobs per day. Presto, HiveQL, Hadoop, and GraphQL are the common query engines over Hive. Hive has been open sourced, but the open source version is different to the one used internally within Facebook.
- **Presto:** a distributed SQL query engine. Although open sourced, Facebook uses a different version internally.
- **unidash:** an internal dashboarding tool. Engineers can create monitors/alerts/graphs with ease, connecting any number of backend data sources. This tool was beyond anything available at the market up to around 2020. These days, [Honeycomb](#) or [Datadog](#) are commercial vendors that come closest.

- **Internal manager tools for code metrics:** both engineers and managers have access to internally built tools to visualize code-level metrics like lines of code (LOC), diff count (basically, number of pull requests or PRs). People can add others as a way to compare stats with their own numbers.

A few interesting notes:

- **Google Docs:** the company-mandated document system. Facebook changed over from **Quip**, which it used for many years. Software engineers I talked with liked Quip somewhat better than Docs. Rumors are that the change was pushed by non-engineers, who found Quip more limiting than Google Docs.
- **Slack** is used by Instagram, but not by Facebook teams.
- **Project management tools:** for simple things, teams choose **Phabricator**. **Tasks** is another popular internal tool that Facebook keeps improving, and many teams are moving over to it. Some teams use **Asana**, but usage is going down across the company in favor of Tasks. Asana was used from the early days at Facebook, which should not be surprising as Asana's founder and CEO, Dustin Moskovitz, is Facebook's cofounder.

Open source

From the outside, it seems clear Facebook is huge on open-source. It has a dedicated [open-source landing page](#) for sharing its commitment to open-source:

"Facebook is committed to the notion that open-source not only provides great technology for developers, but also brings the best out in people."

Facebook also has open-source technologies that are used across the industry. The most notable projects are:

- **React:** one of the most popular JavaScript frameworks.
- **React Native:** one of the most popular frameworks to build iOS and Android cross-platform applications.
- **Jest:** a popular JavaScript testing framework.

- [PyTorch](#): a Python-based machine learning framework for dynamic neural networks.
- [Docosaurus](#): building beautiful documentation from Markdown.
- [Rocksdb](#): a persistent and fast key-value storage.
- [Buck](#): a fast mobile build system.
- [Hive](#): a petabyte-scale data warehouse using Hadoop, developed by the Data Infrastructure Team at Facebook. Facebook uses a different version internally. [More details](#).
- [GraphQL](#): a query language for APIs.
- [Presto](#): a distributed SQL query engine. Facebook uses a different version to what is open sourced.
- ... and [many other projects](#).

On the inside, the open-source projects are less mentioned. Facebook tends to use its own version of many open source projects, making it frequently seem to engineers that the open-sourced project is just a side effect of forking other projects.

A good example of a forked project where Facebook used a different version – and was years ahead – is coding and ticketing system [Phabricator](#). Facebook open-sourced this tool which was used by some organizations, including at Uber. Phabricator was maintained for years by former Facebook engineer [Evan Priestley](#) outside of Facebook [up to June 2021](#).

Company divisions

The company consists of these major divisions:

- Facebook
- Messenger
- Whatsapp
- Instagram
- Meta Reality Labs

Each organization is different in culture. Individual culture heavily depends on the organization's leadership.

Whatsapp was famous for having a different culture and processes to the rest of Facebook. For example, they used to have different performance review cycles up to around 2020. Whatsapp also placed more importance on code quality compared to the "move fast and break things" culture of core Facebook.

Company-wide processes relevant for engineers and EMs

The weekly company all-hands meeting on Fridays has been a tradition at Facebook since its early days until today. Zuck and his executive team attend these meetings and employees are encouraged to ask candid questions. This meeting is an hour long, weekly.

In the past, when most Facebook employees were in the Menlo Park HQ, many of the questions were asked live by the audience. As of late, most questions are submitted via a polling feature in Facebook Workspace, with employees voting for questions, and the most popular ones being asked. Questions can be submitted anonymously.

The weekly all-hands still being in place is notable, as almost all similar sized companies have dropped such events, or reduced them significantly. For example, Google's weekly all-hands have been reduced to monthly, with the focus only on product and business strategy. Google made this change in 2019, with CEO Sundar Pichai partially blaming leaks for this change [when he reduced the meeting frequency](#). Amazon [has no such all-hands in place](#).

The fact that Facebook/Meta, even at close to 70,000 employees, is sticking with the format is a notable signal of transparency.

Zuck Review is a meeting where a product idea is demonstrated to Zuck, who gives feedback on the spot. Positive feedback typically makes it easier for engineers and managers to attract others to join the project.

The reviews were an important part of how Facebook developed products and features in its early years. Here's how an intern at Facebook described his Zuck Review

experience in 2012:

"Our hackathon was selected to attend THE Zuck Review. Only 6 hackathons were to be presented in that review. (...)

We got into the meeting, met Zuck, Schrep and a few other higher-ups. We nervously started our presentation when suddenly Zuck stopped us.

'Wait, can you introduce yourself first?'

Then after the introduction, we continued with the presentation (in which I embarrassed myself because my buddy had to set up my laptop for it). We were about to ask for a photograph after the presentation, but both of us were too shy to ask. By the end of the presentation Zuck had only said these 5 words to us: "Cool, we should ship this."

The lack of direct headcount allocation to teams is one of the most curious processes at Facebook. The company does have headcount planning and budgeting in-place, allocating headcount to teams. Team headcount is referred to as PIDs - Position IDs.

Instead, the below four pillars ensure that engineers naturally distribute themselves on projects that are the most important for the company:

1. **The focus on impact:** engineers join projects which have enough impact potential.
2. **Frictionless internal mobility:** it's not only easy to move, but also encouraged to do so within the company with no friction. If a project either won't have impact or the engineer doesn't feel supported, they just move.
3. **Bootcamp distributing incoming hires:** new engineering hires are typically not allocated to teams, save for a few key, experienced hires. New engineers talk with multiple teams, and it's down to the engineering managers and engineers to "sell" new hires for their teams.
4. **Teams can only take on engineers while they have PIDs left.** Once a team fills its open PIDs, no new hires can be made and their managers won't advertise

themselves to Bootcamp grads or to internal transfers.

The above approaches mean that projects get funded whose impact is clear and engineers can “claim” that impact as part of their work. Ones where engineering managers cannot sell their project or don’t support managers, are starved of interest.

The approach has its limits, and some strategic projects might get underfunded if they cannot sell themselves, or if engineers don’t think they are important. However, leadership usually corrects this soon enough.

The company does not change its headcount allocation, even for company strategic projects. For top priority projects, leadership naturally wants to staff it quickly. At most new companies, the new hires would just be assigned to it. Yet within Facebook, the most that happens is that bootcampers are encouraged to consider it. It’s common to see senior leadership folks come and tell engineers to explain why this project is of strategic value and what impact it has at the company-level. But nobody would be forced to join it.

As a concrete example of this principle at play, I talked with an early Facebook designer. This designer recalled how, in 2011, Zuck walked up and asked him to go and work on Ads. Zuck explained how he thought this area was strategic, and that he’d noticed this designer was influential enough to bring engineers with them, if they moved over. This designer considered the request, then moved and indeed pulled engineers with them, as they outlined the projects and the impact this team would have on Facebook.

There is no other Big Tech firm which takes a similarly radical approach to headcount priority allocation. The fact that Facebook not only does this but does it successfully, should be reason enough for startups to not always want to follow what other companies do in their processes.

Changes at the company over time

The “engineering-first” culture is slowly fading, old-time Facebook engineers share. An engineer who was at the company 2013-2018 shared:

"The engineering-first culture eroded considerably over the years. It started with the pace and participation of hackathons in the culture.

"The culture eroding is also a by-product of the ratio of engineers being lower and lower every year. The company has been split into multiple divisions, all with slightly different cultures and geographical footprints.

"The ever-aging engineering population also contributed to the change. The initial Facebook leadership core was really, really young when the company started: and the youth could be felt within the whole company."

"Be Open," as a cultural value, is disappearing. Up to around 2018, "Be Open" was a commonly referenced cultural value, and one displayed on posters in the office.

Openness as a value is no longer part of the updated Facebook company values. Employees I talked with attribute this to the nature of leaks in the past. Facebook even has an internal group called "Examples of Facebook's culture moving from open to closed".

The nature of engineering is still very open, with the vast majority of information accessible to engineers. However, organizations are more commonly restricting information to only those in their organization. Still, engineers joining from other Big Tech are surprised how Facebook feels more open than comparable companies.

My view is that full openness and transparency has to go over time, as the company grows to tens of thousands of employees. It's impressive that at Facebook, this value lasted as long as it did.

Privacy changes are much more strictly reviewed and are slower. Following the [Cambridge Analytica scandal](#), all major changes within Facebook go through lengthy and thorough legal and privacy reviews. On one end, this slows down processes. On the other, I'm not surprised such counter-balances are put in place, as Facebook aims to prevent similar privacy leaks or security issues as they have had in the past.

The relationship with testing. The old Facebook had very little automated testing in place. Early engineers shared how people used to also add a revert plan to their diff to

instruct how to undo the change in the frequent case that this needed to be done. This approach has improved over the years with better test tooling. Still, Facebook has a less testing-first approach than many other Big Tech companies.

Performance cycle changes. Used to be every 6 months up until 2021. Starting from 2022: doing it annually.

Planning cycle made longer. Historically, Facebook has been doing planning every 6 months. This is changed to annual planning, starting from 2021. The goal is to think longer term. People within Facebook shared how they expect it will take another few cycles of planning for this year-long approach to stick.

Adding additional engineering axes.

- Adding the People Axis around 2015. This axis was not part of early Facebook. It was added to capture non-technical contributions, team effect, mentoring, interviewing and community contributions. Since 2015, this axis has been extended, e.g. people unblocking others.
- Adding the Engineering Excellence axis. This happened around 2016-17 as the 4th axis.

5. Advice

Advice to succeed at Facebook as an engineer

As the advice is forward-looking, I'll be using the company name "Meta" instead of Facebook in these three advice sections.

- **Don't get intimidated by Bootcamp!** The information overload will be real. However, know that you're not expected to know everything, and the goal is to get a feel for how Meta works, and get to know people and teams. Approach Bootcamp with a mindset of curiosity and wanting to understand things; this will make it a pleasant experience. However, focus on impact here, as well. Instead of understanding how all systems work, focus on shipping something.

- **Set realistic expectations.** Meta is no longer exactly a startup, nor a “move fast” company. Be prepared to join a big company, but one where you as an employee are treated well.
- **Know that impact is very important at Meta, always.** If you manage to consistently deliver outsized impact you'll do well. If you fail to do so: you might get on Below Expectations, and have to worry about PIPs.
- **Know how PSC works and what the inputs and outputs are.** PSC is important for everyone at Meta. Prepare for it, understand how it works so you can make the most of it. Obviously, don't do your work only in hopes of optimizing for a great PSC outcome; but don't be blind to this process either.
- **Don't be afraid to ask for help.** Developing products, technology and people at Meta is a team sport!
- **Take the initiative.** Meta is truly a bottom-up company. You are in charge. Find good mentors. Find good projects where you can perform at your best. If you see an opportunity, don't hesitate to switch teams.

Advice to succeed at Meta as an engineering manager

I asked a long-tenured Director of Engineering, who is currently at Meta, for advice they give new EMs who join. Here is what they shared:

“The two biggest mistakes I see new managers at Meta make are:

“Some assume that an EM position at Meta is just like any other manager position.

It's not! Pay attention to the differences in culture, which can be significant compared to other companies, including Big Tech. Get a sense of what the expectations for managers are.

“Even more EMs assume that Meta is so different that your past manager experiences don't matter. Curiously enough, more managers make this mistake of disregarding their experiences, as they get overly intimidated by the company.

“In reality, the truth is in the middle. One definitely needs to learn about the Meta way of managing teams. The emphasis being on:

- Growing engineers
- Collaboration
- Giving people a lot more freedom than at other places
- Learning complex HR processes.

"Generally, if you have been a strong manager at your previous roles, you are highly likely to be a strong manager at Meta.

"Finally, learn to sell your team. This is something all managers at Meta have to learn rapidly. Any given Bootcamper can have dozens of teams to reach out to them. Managers quickly learn to sell their teams and projects. You won't have any team if you cannot sell it!"

Advice when leaving Meta

Leaving Facebook is harder than it sounds. This is not just because of the very top-of-the-market compensation, which makes it hard to get offers that even match Facebook earnings. It's also because the custom tech stack and non-conventional practices can make it harder to secure a software engineering position at other companies.

Here is advice that engineers and engineering managers who left Facebook for other Big Tech companies and startups shared:

- **Refresh your knowledge of "standard" industry tooling.** For example, if you're a backend engineer, it's worth playing around with things like Git, AWS, observability tools in the "outside". Be conscious how custom the Facebook tooling is, and how, in some cases, these tools are behind where the industry is at. This is especially true if you'll want to leave to work at startups which overwhelmingly use the "common" tooling in the industry.
- **Know that most other places test their code much more.** Refresh this know-how. Former Facebook hiring managers shared how they had to reject some applicants from Facebook - especially during coding interviews or takehome exercises - as they did not test their code, and missed the mindset of double checking if their approach was correct.

- **Expect much less custom tooling and frameworks at startups, and for good reason.** Facebook has built most of their tooling in-house, which was a sensible choice for their use case. They built one of the world's largest website and apps. There were few, if any, solutions that met their needs. However, the tech landscape has changed since Facebook started. Most startups are better off buying over building. Be careful if you are tempted to build something inside a startup that you observed Facebook build themselves.
- **Be wary of habits you bring with you from Facebook, like talking up your own work.** An engineering manager working at a scaleup in London shared how former Facebook engineers they hired caused a lot of contention at the workplace. This was because they brought the "bragging via Workplace posts" with them to the new company, and sent out writeups of their work and its impact in a way that was considered over the top at this company.

Advice for founders and managers on inspiration to take from Facebook

Facebook is one of only two companies globally to grow to roughly every second human being on the planet using their products, within 20 years of the company starting. The other is Google.

What are things that founders and managers working elsewhere can take as inspiration from Facebook? My thoughts:

- **Beware of survivor bias in all practices.** Facebook succeeded not mainly because of its practices, but in good part because of the business environment and business decisions. For example, there was no global social media network when it started, mobile phones were just launching and smart acquisitions like Instagram. In some cases, it might have succeeded despite some of its own practices.
- **Don't go cargo cult.** Copying a practice that Facebook uses in the expectation that you'll achieve progress in your own workplace, isn't enough on its own. The [term "cargo culting"](#) derives from World War 2, when some island communities in the Pacific region were observed to perform rituals intended to summon US

military personnel to deliver resources. For example, they mimicked the dressing and day-to-day activities of the soldiers after those soldiers left, expecting that they would receive supplies.

- **Know how the type of people Facebook hires differs from your company.** Facebook deliberately pays the top 90th percentile of the market – sometimes even higher. They have high expectations of people, but compensate accordingly. Chances are, your company won't have this budget. Keep in mind that this means that Facebook can ask more from their engineers, and might need to motivate them less. The quality of their engineers might also mean they can get away with practices most other companies would suffer from.
- **Caution in following Facebook's documentation and testing practices.** Facebook has less documentation and testing in-place than most tech companies. This approach works for a few reasons: historic ones, engineers having long tenures, Facebook hires some of the best engineers, they built systems to counter the lack of documents and testing. I would not suggest deliberately following this approach, especially not in the age of asynchronous and remote work.
- **Take inspiration from the transparency that the "Workplace posts" culture Facebook brings.** Facebook runs on Workplace, and this approach heavily encourages engineers to articulate and share the impact of their work to a large, internal audience. This approach results in information flowing more freely, and in [software engineers becoming better writers](#). I am not suggesting you should onboard to Facebook Workplace. However, encouraging an environment where individual contributors verbalize the impact of their work, write it down, and share it: this can help with information flow and innovation within the company.
- **Facebook still has a founder CEO.** Mark Zuckerberg has been central to the culture of the company, and to keeping this culture on track.
- **An engineering-centric culture.** Engineers would struggle to find similar autonomy and appreciation elsewhere. Though not the only approach to building a successful tech company, Facebook would have surely missed out on lots of innovation if it did not give autonomy to engineers.
- **Scaling culture is possible, up to a point.** Facebook's culture is distinct from any other Big Tech company, and is closer in spirit to early Facebook than a

stereotypical, faceless corporate culture. Facebook managed to scale its culture with major investments like Bootcamp, but also by continuing to reward impact, consistently paying top of market, and granting large autonomy to engineers and teams.

- **Are processes, documentation and testing overrated?** Facebook engineers have fewer processes, document less and test less than most of Big Tech. And yet, the company has grown to serve more users than almost any other company. Does this mean processes are overrated? No, but it does mean that copying the current incumbents in the tech industry is not always the best strategy for a startup.
- **Radically different businesses can succeed with radically different approaches.** Facebook had a huge number of users for every engineer from the very beginning, which was different to most businesses. Instead of investing in testing, they invested in logging, operational awareness, automatic canarying and alerting on canarying. It changed how it did engineering, and used the advantages it had, such as a huge user base for its service, but without the high reliability bar of, say, a financial service.
- **Custom onboarding to solve for custom pain points.** No other tech company has invested as much into onboarding as Facebook has with Bootcamp. This investment pays off, given how exotic the tech stack and tooling are. Make no mistake, Bootcamp is a very expensive type of onboarding. Still, Facebook decided it is well worth the cost in people's time and staffing.
- **As the company grows, incentives need to change.** Facebook kept responding to its growing pain points by tweaking incentives for employees. The most notable changes have been the introduction of the new axes - People and the Engineering Excellence ones - and how the openness culture disappeared over time. Any company that grows as fast as Facebook does needs to continuously change.
- **Embracing internal mobility instead of fearing it.** Many companies fear creating overly loose internal mobility guidelines because they worry that some of their key teams might have engineers leave for other teams. But the result is often that people just quit the company, if changing teams is harder than interviewing externally. Facebook took the opposite approach: making moving much easier. By making engineers responsible for their own impact, they also ensured that

impactful areas always have enough staffing. This system conveniently helps identify struggling teams and managers, and leadership can step in and take action.

I caution against blindly copying processes just because a company like Facebook does them. My view is it's helpful to understand the mechanics that make companies successful. At the same time, don't forget that your environment, the people around you, and your business context is different.

Instead of copying an approach from Facebook – or another company – find your biggest pain points, and solve them. Reach for the toolset which might include approaches that Facebook and others have used, but use them only if you reasonably believe they will help ease your biggest problems.

Summary

[Part 1](#) and this Part 2 have gone into considerable breadth and depth into how Facebook works from an engineering perspective, and why it works as it does.

I caution blindly copying any practices Facebook has. However, I do recommend thinking about *why* certain practices might have helped the company, and what are practices where Facebook succeeded despite putting them in place.

I hope this deepdive was useful. For another engineering culture deepdive, see [Inside Amazon's Engineering Culture](#).

If you're a current or former Meta employee and spot something incorrect, please email newsletter@pragmaticengineer.com or leave a comment.

Thanks to the many current and former Facebook engineers and managers for their help and reviewing of this piece.

How did you like this week's newsletter? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)



86 Likes

13 Comments



Write a comment...



AGoogler Jan 14 Liked by Gergely Orosz

New numbers for 2022 (which could be updated in 2023):

Target equity refresh:

E3: 48k, E4: 88k, E5: 175k, E6: 274k, E7: 480k, E8: 850k

Also the multipliers changed for GE and RE: 165% and 250% (down from 200% and 300%).

Besides I don't think the company multiplier is applied to the equity refresh (only on the bonus)

LIKE (3) REPLY ...



Guy May 27, 2022 Liked by Gergely Orosz

Do you know where I can find more detailed information on facebook's E/M ladder including competencies, etc?

LIKE (2) REPLY ...

1 reply by Gergely Orosz

11 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing